

Introduction to Programming

ENS-Louis Lumière

David Poirier-Quinot

Introduction to programming

Content

- Programming as a tool for audio engineers
- Three days (3x6h)
- Selected language: **Python**
 - high level
 - simple syntax
 - versatile
 - large community

Data Types

```
s = "hello" # string  
a = 1 # int  
b = 0.1 # float  
c = True # boolean  
list = [1, 2, 3] # list, can hold anything, ordered  
set = {1, 4, 6, 7} # set, unordered, unique elements  
tuple = (1, 2, 3) # tuple, same as list but can't modify  
dictionary = {"apple", 3, "orange", 2} # dictionary, can hold anything, unordered  
z = None # undefined
```

Symbols

Operators

- Addition `+`
- Subtraction `-`
- Multiplication `*`
- Division `/`
- Floor Division `//` : divides and returns the integer part of the quotient
- Modulus `%` : returns the remainder of division
- Exponentiation `**` : raises the left operand to the power of the right operand

Functions

Using existing functions

```
res = 3.14  
print(res) # print to console
```

This code will print the value of `res` in the console.

- `print()` is a **function**.
- A function can accept any number of **arguments** (here just one: `res`).

Some functions return a result

```
res = round(res) # return rounded number: 3
```

Script 1 - First steps

Instructions

Go to <https://pythononline.net>. Create a script that adds two numbers and prints the result using the `print()` function.

Takeaways

- Run script
- Simple operation
- Print to console (+ cast to string)
- Debug workflow

Symbols

Comparison operators

- Equal `==`
- Not equal `!=`
- Greater than `>`
- Less than `<`
- Greater than or equal to `>=`
- Less than or equal to `<=`

Conditions

Conditions

The `if/else` statement is used to do different things depending on a condition:

```
price = 12

if price < 10:
    print("I'll buy it")
else:
    print("not a chance")
```

Conditions

Conditions

`if/else` can become `if/else/elif` to add further conditions:

```
price = 12

if price < 10:
    print("I'll buy it")

elif price < 15:
    print("can you give me a discount?")

else:
    print("not a chance")
```

Code syntax

Indentation

Python uses indentation (either a tabulation or 4 spaces, see pep8) to delimitate blocks of code.

```
if( show_advice == True ):
    # only printed if condition met
    print("don't touch anything or the floor is lava")

# always printed
print("welcome to the Cave of Wonders")
```

Modules

Importing modules

A module is Python code (e.g. from another file) that can be imported in a your code using:

```
import random
```

Module functions

One usually imports a module to make use of its associated functions using the syntax:

```
import random  
r = random.random()
```

Script 2 - Head or tail

Instructions

Create a script that picks a random float between 0 and 1 using the `random()` method of the `random` module. Round that float to an integer using the `round()` function. Using an `if / else` statement, print "head" if the final number is a 0, "tail" if it's a 1.

Takeaways

- Import a `module`
- Generate a random `float`, round to nearest integer
- Use an `if / else` statement
- Compare two numbers using `==`

Loops

For loop

The `for` loop is used to iterate over elements in a sequence:

```
# init list
fruits = ["apple", "banana", "pear"]

# print elements in list
for fruit in fruits:
    print(fruit)
```

Script 3 - Dice game

Instructions

Create a script that throws two 6-sided dices, a blue and a red, 10 times, using the `randint()` method of the `random` module (check e.g. <https://docs.python.org> for the API). Print the success rate of the blue dice.

Takeaways

- Use a `for` loop
- Use a variable to keep track of system state
- Generate a random integer

Loops

While loop

The `while` loop is used to repeat a block of code until a condition is met:

```
# init counter
counter = 1

# loop until condition met
while( counter < 10):

    counter = counter + 1
    print(counter)
```

Script 4 - Number guessing game

Instructions

Create a script that picks a random number between 1 and `max_number` (e.g. 10). In a `while` loop, prompt user to guess that number. print "higher" if user guess is too low and "lower" if it is too high. Report success and exit the `while` loop if the user guessed right.

Takeaways

- Require user input
- Cast `string` to `int` using `int(my_string)`
- Use a `while` loop

Code syntax

Commenting code

Comments are here to **help you/others understand the code**. Suggest a "header" comments for each "paragraph" of code, in english, with consistent and evocative wording.

```
# init locals
score, num_trials, num_faces = 0, 10, 6

# loop over trials
for iTrial in range(num_trials):

    # throw dice
    dice_value = random.randint(1, num_faces)

    # increment score
    score += dice_value
```

Code syntax

Useful Python syntax

```
# +=, *=, etc.  
score += 1 # instead of score = score + 1  
  
# one line loop (aka "comprehension")  
my_list2 = [x*2 for x in my_list]  
  
# f-strings  
print(f"the score is: {score}") # or even {score:0.1f}  
  
# assign multiple values  
x, y = 10, 20
```

See <https://realpython.com/cheatsheets/python> for more.

Setup programming environment

Guidelines

- An environment **adapted to your needs**
- Should enable quick run/debug iterations (+ variable inspection)
- **Autocompletion** with access to API (application programming interface)
- Enables updates without breaking retro-compatibility (e.g. package installs)
- Large screen real-estate + window manager
- Knowledge of **main shortcuts**

Setup programming environment

Manual install (recommended)

- Install a package manager: `homebrew` , `aptget` , `chocolatey`
- Install an IDE (Integrated Development Environment): `VSCode` , `VSCodium`
- Install a Python distribution manager: `pyenv`
- Setup IDE to use installed Python

Automatic install: all-inclusive IDE (easier, less flexible)

- Install `Spyder` (<https://www.spyder-ide.org>)

Setup programming environment

Virtual Environment

- Multiple projects -> different Python versions and packages -> Interferences.
- Virtual environment: an utility to install/run python at folder scale.
- `requirement.txt` : a file shipped with a project listing the required python packages
When to install in virtual env.
- Virtual environment created using the `venv` utility
- `pyenv` installed earlier uses virtual environments

Vibe coding and the use of LLMs

Double-edged sword

- + Makes you more efficient
- + Increases your reach
- No more journey, directly reach destination. Fosters **apathy for your own projects**
- No long term build-up skill, **instant red flag** during interviews

Like cheat-codes in games

- Use once you've completed "the game"
- Maybe use it for stages utterly useless. Not boring, useless: neither useful nor fun
- Even then, sparingly or you'll soon stop playing

Checkpoint

QuickScript 1 - Search Array (10 min.)

Instructions

Manually find the smallest value of an array of 50 random integers in [1:100]:

```
numbers = [random.randint(1, 100) for x in range(50)]
```

and the index of that value, **without** using any built in function such as:

```
min_value = min(numbers)  
min_index = numbers.index(min_value)
```

Script 5 - Audio file player

Instructions

Create a script that loads the audio file `./assets/drumloop.wav` and plays it using the `read` and `play` methods of the `soundfile` module.

Takeaways

- Install package
- Construct path
- Load file from disk

Script 6 - Music score player

Instructions

Create a script that retrieves all the names of all the `.mp3` files in `./assets/scales` and plays each of them (`for` loop) using the `pygame` module.

Next, create a score using a `dictionary` that stores an array of `notes` (indices of audio files) and `ioi` (inter onset interval, unique float). Play the score. Use the `sleep` method of the `time` module to pause between notes.

Takeaways

- List content of a folder
- Use a `dictionnary`

Script 5 - Tap delay reverb

Instructions

Create a script that loads the audio file `./assets/drumloop.wav`. Create a `numpy` array in which you will copy the content of the audio file twice: once starting from index 0, once starting from an index that corresponds to a 100ms delay. Play the resulting audio.

Modify the script to handle any arbitrary number of these "tap delays" using a `for` loop. To configure your reverb, use a dictionary with keys `delays` (list) and `gains` (list). Create various reverb configs.

Takeaways

- Use `numpy` arrays
- Manipulate audio content (delay line)

Overview of programming languages

Different syntaxes and philosophies

Differences **hard to summarize in a few slides**.

To get a genuine understanding, design the same application (e.g. read file, sort content) using each of these (no particular order):

- C, C++, Rust
- Python, JavaScript/TypeScript
- Java, C#, Swift
- Matlab/Octave, Julia

Overview of programming languages

Which language for which application?

- Performance needed: C, C++, Rust
- Fast development / prototyping: Python, JavaScript
- Web application: JavaScript / TypeScript
- Data science / scientific computing: Python, MATLAB, Julia
- Mobile application: Swift, Kotlin
- Embedded / hardware control: C, C++
- Games / Interactive apps: C++ (Unreal), C# (Unity)

Definitions

- **Library** (aka **module** in Python)
 - A collection of reusable functions/classes that your code calls
 - Examples: NumPy, SciPy, Matplotlib
- **Framework**
 - A larger structure that calls your code
 - Defines how the application is organized
 - Examples: Django, Qt, Unity, AngularJS

Library: you control the flow

Framework: it controls the flow

Definitions

- **Workflow**
 - A sequence of steps used to accomplish a task
 - Example: load audio -> process -> analyze -> export
 - Reflect on your workflow to organise your code base / folder structure.

Script 7 - Convolution reverb

Instructions

Create a script that loads the audio file `./assets/drumloop.wav` and the impulse response `./assets/ir_mono.wav`. Convolve both using `fftconvolve` from the `scipy.signal` module and play the resulting audio.

Modify the script to detect mono/stereo impulse responses and apply the convolution accordingly. Test with both `./assets/ir_mono.wav` and `./assets/ir_binaural.wav`.

Takeaways

- Use convolution
- Mono / stereo channel manipulation

Script 8 - Additive synthesis

Instructions

Create a script that creates and adds 3 sine waves together (various frequencies and gains) and plays the result.

Plot the resulting signal in the time domain using `matplotlib.pyplot`. Get its frequency content using `np.fft.rfft` and plot it.

Takeaways

- Create and combine audio signals
- Plot an audio signal
- Use an `fft` (fast fourier transform) and plot the frequency content of an audio signal

Script 9 - Audio filters

Instructions

Create a script that loads the audio file `./assets/drumloop.wav`. Create a lowpass filter with a cutoff frequency of 300Hz using the `butter()` method of the `scipy.signal` module. Filter the audio signal and play it. Save the filtered signal to the disk as a `.wav` using the `write()` method of the `soundfile` module. Display the frequency content of the filtered signal.

Takeaways

- Create and apply a filter
- Observe the impact of a filter on frequency content
- Save audio file to disk

Checkpoint

QuickScript 2 - Play audio backwards (10 min.)

Instructions

Create a script that loads `./assets/drumloop.wav` and plays its content backwards.

Object oriented programming

Function

A **function** encapsulates an operation.

```
def euro_to_dollar(x, rate=1.16):  
    return x * rate
```

Key words / concepts:

- `def` and `:`
- indentation
- argument `x` (optional), default argument value `rate=1.16`
- `return` (optional)

Object oriented programming

Class

A **class** is a blueprint for creating objects.

```
class Person:

    # constructor
    def __init__(self, name):
        self.name = name # attribute

    # method
    def print(self):
        print(f"name: {self.name}")
```

Object oriented programming

Class

Key words / concepts:

- `self`
- constructor `__init__`
- **attributes** and **methods** (i.e. functions)

What we call an object is an instance of a class:

```
student = Person("John")  
student.print()
```

Naming scheme

As many conventions as there are languages / teams / devs. [Google Python Style Guide](#):

```
my_variable = 2  
  
def my_function  
  
class MyClass  
  
GLOBAL_CONSTANT = 3.14
```

A good naming scheme makes **reading code more intuitive**.

Script 10 - First class (1/2)

Instructions

Create a function `should_i_win(luck)` that return `True` or `False` depending on the `luck` argument in 0:1. give `luck` a default value of 0.5. Test the function.

Create an `Enemy` class that has one float attribute `health`. Give it a method `take_damage(damage)` that subtract `damage` from `health`. Test the class.

Takeaways

- Create a Function
- Create a Class
- Manipulate Class attributes and methods

Object oriented programming

Class Inheritance

Inheritance allows to define a class that **inherits methods** and **properties** from another class.

- The **parent** class is the class being inherited from
- the **child** class is the class that inherits from another class

Object oriented programming

Class Inheritance

```
class Student(Person):  
  
    # constructor  
    def __init__(self, name, id):  
        super().__init__(name) # call parent constructor  
        self.id = id  
  
    # re-define method  
    def print(self):  
        print(f"name: {self.name}, id: {self.id}")  
  
    # new method  
    def is_registered(self):  
        return id != None
```

Script 10 - First class (2/2)

Instructions

Create the child class `AnnoyingEnemy` that inherits from the class `Enemy`. Give its constructor an additional attribute: `luck`. Modify its `take_damage()` so that depending on the outcome of `should_i_win(self.luck)` the enemy either takes damage or invokes its `boast` method, printing something nasty to the console.

Takeaways

- Implement inheritance
- Use parent methods
- **Override** parent methods

Script 11 - Create module

Instructions

Create a new folder named `my_module`. In it, create a Python script defining the `Enemy` and `AnnoyingEnemy` classes from the previous script. Still in the `my_module` folder, create an empty `__init__.py` script.

Next to the `my_module` folder create a script that imports the module, instantiates the `Enemy` and `AnnoyingEnemy` classes and uses their methods.

Takeaway

- Create a module folder with `__init__.py`
- Import a class from another file (towards architecture organisation)

Script 12 - Audio synthesiser

Instructions

Create a Synthetiser class that can synthesise and play a note using its method `play_note()` that accepts a frequency as argument.

Add a second wave shape option using the `sawtooth` method of the `scipy.signal` module. Allow the syhtnetiser to generate waveforms composed of several harmonics. Add an envelope generator, use it to shape the note played.

Takeaways

- Full fledge class design and implementation

Code organisation

When do you need to create a function?

- When an operation is repeated twice or more at different places in the code (**sanity**)
- When a segment of code does one "thing" that can easily be conceptualised / named (**ease interpretation**)

When do you need to create a Class?

- When you identify a data structure or conceptual object that has its own properties (**attributes**) and behaviour (**methods**).

Classes encapsulate code into conceptual objects, very human-friendly. They make code easier to read, understand, manipulate and reuse.

Code syntax

How to name things

- Variables: be **descriptive**, consistent, not too long
- Functions/Methods: opt for **verb-noun pairs** to clearly indicate functionality
- Class: a simple name that evokes what it is **and what it is not**.

Very important skill

Bad naming will result in increased **technological debt**, very fast

Bad naming fosters "API misinterpretations" that will cost you hours (and hairs).

Script 13 - First UI

Instructions

Create a script that generates a 200x100 window using the `tkinter` module. Change the title of the window to "hello world". Add a label to the window that says "hi there".

Takeaways

- First steps with `tkinter`
- Create a `tkinter` window and `label`

Script 14 - Random number generator

Instructions

Create a script that generates a `tkinter` window with a `button` and a `label`. Pushing the button generates a random number in `[1:100]` displayed by the label.

Takeaways

- Create a `tkinter` `button`
- Associate a callback (`command`) to a button

Script 15 - Best near zero

Instructions

Create a script that generates a `tkinter` window. Add a "Start" `button`, when pressed it starts a countdown timer in a `label`. Pressed again it stops the displayed timer. The player's objective is to stop as close to 0 as possible. Add a `label` to keep track of the best score.

Takeaways

- Use `tkinter` window `after()` method to create UI auto updates
- Use `global` variable declaration to modify its content in functions
- Change button callback (`command`) and text
- First steps into step machines

Version Control

Git

Git is a **command line utility** to create version control over a set of scripts. It acts as a **time-machine** on steroids for devs.

```
git add modified_script.py  
git commit -m 'description of the modification'
```

GitHub

GitHub (<https://github.com>) is a **remote-hosting server** based on git to store, share and collaborate on projects.

Version Control

Setup Git and GitHub

- Install git on your machine
- (optional) install a GUI for git
- Create an account on GitHub

Init a Git repository

- Create a `git_test` folder with a `readme.txt` and a python script in it
- Add git tracking to that project
- Commit both files

Version Control

Setup a GitHub repository

- On GitHub, create a new project
- Setup `git_test` git to use that repository as main remote
- Push your local project onto GitHub

Commit and Push

- Change something in your python script
- Commit change
- Push on remote

Version Control

Pull

- Clone the repository from another student onto your machine

Additional features

- Share read/write access
- Stash / show / discard changes
- Checkout former states
- Branches
- Pull Requests

Checkpoint

QuickScript 3 - Dino Game

Instruction

Open the project `./quickscrip3/3_dino`. Init a git repository and commit everything. Apply the following modifications:

- Add looping ambient `Sound/ambient.mp3` at volume 0.3
- Make Ptera spawn as often as Cactus, a third of them high enough not to be a threat
- Make Clouds spawn as often as enemies, make them move 4 times faster
- Replace triple Cactus by double Cactus (without modifying files)

Show implemented changes using

```
git diff main.py objects.py
```

Mini Project

Instructions

- Imagine a piece of code that would be useful and/or interesting to implement
- Write description, imagine architecture
- Submit concept for validation
- Implement

Takeaways

- Autonomous conception and implementation
- On the fly codebase and objectives adjustments based on tests